

Department of Computer Engineering  
University of Peradeniya  
CO523 - Programming Languages  
Lab 07: Subprograms, Parameter Passing, and Stack Frames

E/20/449

Wijewardhana A.R.S.S.

## Task 01

### Python

```
# Task 01 - Python (pass-by-value behavior with immutable int)
def modify(x):    x = x + 10
print("Inside function:", x)
a = int(input("Enter an integer:
")) print("Before function:", a)

modify(a)
print("After function:",
a)
```

```
Enter an integer: 10
Before function: 10
Inside function: 20
After function: 10
```

### Explanation: Pass-by-Object-Reference in Python

- **Function Call:** In the Python program, the value of a is passed to the function modify(a).
- **Internal Modification:** Inside the function, the parameter x is increased and printed, which shows a changed value locally.
- **External Consistency:** After the function execution finishes, the value of a printed outside remains the same.
- **Immutability:** This occurs because integers in Python are **immutable**; modifying x inside the function creates a new object rather than changing the original variable a.

## C

```
// Task 01 - C (pass-by-value)

#include <stdio.h>
void modify(int x) {    x = x + 10;
printf("Inside function: %d\n", x);
} int main() {    int a;
printf("Enter an integer: ");
scanf("%d", &a);
    printf("Before function: %d\n",
a);

modify(a);
    printf("After function: %d\n",
a);    return 0;
}
```

```
Enter an integer: 12
Before function: 12
Inside function: 22
After function: 12
```

### Explanation: Pass-by-Value in C

- **Function Call:** In the C program, when `modify(a)` is called, the function receives a copy of the variable `a` in the parameter `x`.
- **Internal Modification:** Inside the function, `x` is changed and printed, so the output shows the updated value locally within the function's scope.
- **External Consistency:** When printing `a` in `main()` after the function call, the value remains unchanged because only the local copy `x` was modified.
- **Core Principle:** This demonstrates that C uses **pass-by-value** by default, meaning the original variable is protected unless pointers are used to pass a memory address.

## Task 02

### Python

```
# Task 02 - Python (pass-by-reference simulation using mutable list)
def modify_list(lst):
    lst[0] = lst[0] + 10
    print("Inside function:", lst)
    numbers = [int(input("Enter an integer:
"))] print("Before function:", numbers)

modify_list(numbers)
print("After function:",
numbers)
```

```
Enter an integer: 25
Before function: [25]
Inside function: [35]
After function: [35]
```

### Explanation

#### Explanation: Pass-by-Object-Reference with Mutable Types

- **Input Storage:** In the Python program, the user input is stored inside a list (`numbers = [value]`).
- **Function Call:** This list is passed to the function `modify_list(numbers)`.
- **Internal Update:** Inside the function, `lst[0]` is updated by adding 10, so the output shows a changed value inside the function.
- **Persistent Change:** After the function ends, the value remains changed when printed outside.
- **Mutability:** This occurs because lists in Python are **mutable**, meaning the main program and the function are both working on the same list object in memory.

```
#include <stdio.h>
void modify(int *x) {    *x = *x + 10;
printf("Inside function: %d\n", *x);
} int main() {    int a;
printf("Enter an integer: ");
scanf("%d", &a);
    printf("Before function: %d\n",
a);

modify(&a);
    printf("After function: %d\n",
a);    return 0;
}
```

```
Enter an integer: 26
Before function: 26
Inside function: 36
After function: 36
```

## Explanation

### Explanation: Pass-by-Reference via Pointers in C

- **Pointer Reception:** In the C program, the function `modify()` receives a pointer (`int *x`), which is the memory address of the variable `a`.
- **Address Passing:** When calling `modify(&a)`, the address of `a` is passed rather than a copy of its value.
- **Direct Modification:** Inside the function, the operation `*x = *x + 10` modifies the value stored specifically at that memory address.
- **Persistent Update:** The output shows the updated value inside the function, and it remains updated when printed in `main()` after the function call.
- **Core Principle:** This demonstrates **pass-by-reference** behavior in C, achieved by using pointers to manipulate the original variable's memory directly.

## Task 03

```
def function_c(z):    print(f" → ENTER
function_c(z={z})")    result = z * 2
```

```

    print(f"    Computing: {z} * 2 = {result}")
print(f" ← EXIT function_c, returning {result}")
return result

def function_b(y):
    print(f" → ENTER function_b(y={y})")
    print(f"    Calling function_c({y + 1})...")
    result = function_c(y + 1)    final = result + 5
    print(f"    Computing: {result} + 5 = {final}")
    print(f" ← EXIT function_b, returning {final}")
    return final

def function_a(x):
    print(f" → ENTER function_a(x={x})")
    print(f"    Calling function_b({x * 2})...")
    result = function_b(x * 2)    print(f" ← EXIT
function_a, returning {result}")    return result

final_result = function_a(3)
print(f"\nFinal result: {final_result}")

```

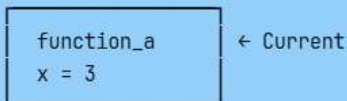
```

→ ENTER function_a(x=3)
    Calling function_b(6)...
→ ENTER function_b(y=6)
    Calling function_c(7)...
→ ENTER function_c(z=7)
    Computing: 7 * 2 = 14
← EXIT function_c, returning 14
    Computing: 14 + 5 = 19
← EXIT function_b, returning 19
← EXIT function_a, returning 19

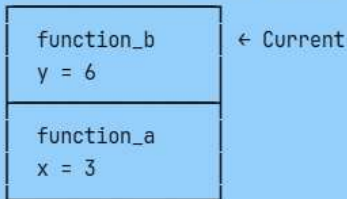
```

Final result: 19

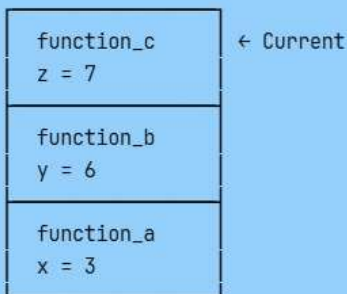
Stage 1: function\_a(3) called



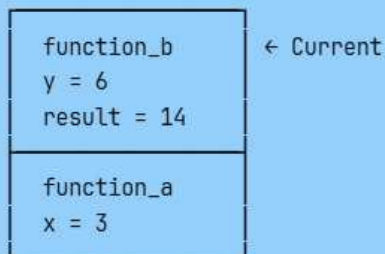
Stage 2: function\_b(6) called from function\_a



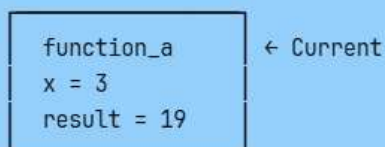
Stage 3: function\_c(7) called from function\_b



Stage 4: function\_c(7) returns 14



Stage 5: function\_b(6) returns 19



## Explanation

This output shows how nested function calls execute using the call stack (LIFO). First, `function_a(3)` starts and calls `function_b(6)` (because  $3 * 2 = 6$ ). Then `function_b(6)` calls `function_c(7)` (because  $6 + 1 = 7$ ). Inside `function_c`, the computation  $7 * 2 = 14$  is done and it returns 14, so its stack frame ends. Control goes back to `function_b`, which then computes  $14 + 5 = 19$  and returns 19, ending its frame. Finally, `function_a` receives 19, returns it, and the program prints Final result: 19.

## Task 04

```
def factorial(n, depth=0):    indent = "  " * depth
print(f"{indent}→ CALL factorial({n}) [Depth: {depth}]")
    if n <=
1:
        print(f"{indent} Base case
reached!")    print(f"{indent}← RETURN
1")    return 1
        print(f"{indent} Recursive call: factorial({n-1})")
result = n * factorial(n - 1, depth + 1)    print(f"{indent}
Computed: {n} * {result // n} = {result}")
print(f"{indent}← RETURN {result}")    return result

result = factorial(5) print(f"\nFinal
Result: 5! = {result}")
```

```
→ CALL factorial(5) [Depth: 0]
Recursive call: factorial(4)
→ CALL factorial(4) [Depth: 1]
Recursive call: factorial(3)
→ CALL factorial(3) [Depth: 2]
Recursive call: factorial(2)
→ CALL factorial(2) [Depth: 3]
Recursive call: factorial(1)
→ CALL factorial(1) [Depth: 4]
Base case reached!
← RETURN 1
Computed: 2 * 1 = 2
← RETURN 2
Computed: 3 * 2 = 6
← RETURN 6
Computed: 4 * 6 = 24
← RETURN 24
Computed: 5 * 24 = 120
← RETURN 120

Final Result: 5! = 120
```

## Explanation: Recursive Calls and Stack Overflow

- **Missing Stopping Condition:** Without the base case, the factorial function has no stopping condition, so it keeps calling itself infinitely
- **Memory Growth:** Every recursive call creates a new stack frame, causing stack memory to grow rapidly.
- **Python Safety Limit:** In Python, once the recursion goes beyond the recursion limit (around 1000 calls by default), the interpreter stops the program and raises a `RecursionError`: “maximum recursion depth exceeded”.
- **Program Failure:** The program fails because the call stack keeps expanding until Python forces it to stop or the system runs out of stack space.

## Task 05

### Python

```
def f(a):  
    b = a + 2  
    return b  
    def g():  
x = 3      y  
= f(x)  
return y  
g()
```

- `g()` called- Creates `g` frame (no params, locals: `x=3`, then `y=f(3)`).
- `f(x)` called- Creates `f` frame (param `a=3`, local `b=5`, returns `5`). `f` frame popped.
- `g` returns `y=5`, `g` frame popped.

| Stage               | Function Name | Parameters | Local Variables     | Return Value |
|---------------------|---------------|------------|---------------------|--------------|
| 1 (g call)          | g             | (none)     | x = 3, y = (uninit) | (uninit)     |
| 2 (f call inside g) | f             | a = 3      | b = 5               | 5            |
| 3 (f return)        | g             | (none)     | x = 3, y = 5        | 5            |
| 4 (g return)        | (end)         | -          | -                   | -            |

## C

```
#include <stdio.h>
int f(int a) {
    int b = a + 2;
    return b;
}
int g() {
    int x = 3;
    int y = f(x);
    return y;
}
int main() {
    int result = g();
    printf("Result: %d\n", result);
    return 0;
}
```

Similar to Python, main calls g (frame: locals x=3, y=f(3)). g calls f (frame: param a=3, local b=5, returns 5). Pops f, then g returns 5 to main.

| Stage                | Function Name | Parameters | Local Variables     | Return Value |
|----------------------|---------------|------------|---------------------|--------------|
| 1 (g call from main) | g             | (none)     | x = 3, y = (uninit) | (uninit)     |
| 2 (f call inside g)  | f             | a = 3      | b = 5               | 5            |
| 3 (f return)         | g             | (none)     | x = 3, y = 5        | 5            |
| 4 (g return to main) | main          | (none)     | result = 5          | 0            |